

Milestone-3: Conceptual Questions on CNN, torchaudio and wandb

Kaushal Vaid
24f2000681@ds.study.iitm.ac.in

Feb 26th, 2026

1 Concepts Involved in this Assignment

- PyTorch Tensors (Shapes, Flattening)
- Audio Processing (MelSpectrograms with torchaudio)
- CNN Architecture (Conv2d, MaxPool2d dimensions)
- Training Loops (Loss calculation, Optimizers, Logging)

2 Resources

- PyTorch Tensors: PyTorch Tensor Basics
- Audio Processing: Torchaudio MelSpectrogram Documentation
- CNNs in PyTorch: Training a Classifier (CIFAR-10)
- Loss Functions: CrossEntropyLoss Docs
- Experiment Tracking: Weights & Biases QuickStart

3 Questions and Approaches

Question 1

Problem: You have a batch of grayscale spectrogram images represented as a PyTorch tensor with shape (32, 1, 64, 64). You need to flatten this tensor (excluding the batch dimension) to feed it into a Fully Connected (Linear) layer. What is the size of the input features for that Linear layer?

Options:

- 64
- **4096**
- 2048
- 1024

Approach: I have a batch of grayscale spectrogram images, so the first thing that clicked in my mind is the dimension of the tensor: (batches, channel, height, width). If I flatten the tensor

to feed to FC Layers, the input size will be equal to $C \times H \times W$. So for grayscale, $C = 1$, and $H = W = 64$. Hence the answer is $2^6 \times 2^6 = 2^{12} = 4096$.

Question 2

Problem: You are converting a 1-second audio clip (Sample Rate = 16,000 Hz) into a Mel-Spectrogram using torchaudio. The configuration is `n_fft=400`, `hop_length=160`, `n_mels=64`. Based on your code output, what is the resulting tensor shape?

Options:

- (1, 64, 16000)
- (1, 400, 101)
- **(1, 64, 101)**
- (1, 128, 100)

Approach: I have to convert a 1s audio clip (sample rate = 16000Hz) into a mel-spectrogram using torchaudio. The configuration given is `n_fft=400`, `hop_length=160`, `n_mels=64`. The output tensor, as per my calculations, will be (1, 64, 101).

Question 3

Problem: Your CNN passes a (1, 64, 64) input through `nn.Conv2d(..., kernel_size=3, stride=1, padding=1)` followed by `nn.MaxPool2d(kernel_size=2, stride=2)`. What is the spatial output shape (Height, Width)?

Options:

- (64, 64)
- **(32, 32)**
- (31, 31)
- (16, 16)

Approach: I have a (1, 64, 64) tensor passed into a convolutional layer with `kernel_size=3`, `stride=1`, `padding=1`. After this layer, the transformed tensor spatial dimensions will remain (1, 64, 64). It is then passed into a max pooling layer with `kernel_size=2`, `stride=2`, giving a final spatial shape of (32, 32) for (height, width).

Question 4

Problem: You have a hidden layer defined as `nn.Linear(in_features=128, out_features=64)`. How many total learnable parameters (weights + biases) does this specific layer contain? **Options:**

- 8,192
- **8,256**
- 8,128
- 128

Approach: I am given a hidden layer in a neural network with input features being 128 and output features being 64. My total learnable parameters (weights + biases) will be $(\text{input features} \times \text{output features})_{\text{weights}} + (\text{output features})_{\text{biases}}$. That is $(128 \times 64) + 64 = 8256$.

Question 5

Problem: Your training dataset consists of 1,050 audio files. You initialize a DataLoader with `batch_size=32` and `drop_last=False`. How many batches will run in one epoch?

Options:

- 32
- **33**
- 30
- 32.8

Approach: My training dataset is given to consist of 1050 audio files. I am initializing a DataLoader with `batch_size=32` and `drop_last` set to `False`. The number of batches will be $\text{INT}(1050/32) + 1$ OR $\text{CEIL}(1050/32)$, which is 33.

Question 6

Problem: You are performing a 3-class classification. The model outputs logits $[2.5, 1.0, 0.1]$ for a target of Class 0. Using `nn.CrossEntropyLoss`, what is the calculated loss value (rounded to 3 decimals)?

Options:

- 0.045
- **0.273**
- 1.500
- 0.890

Approach: I am performing a 3-class classification. The model outputs the logits $[2.5, 1.0, 0.1]$ for a target of class 0. Using `CrossEntropyLoss`, the loss value evaluates to 0.273.

Question 7

Problem: You are training with SGD. Current Weight = 0.5, Gradient = 0.2, Learning Rate = 0.01. What is the new weight value after one optimizer step?

Options:

- 0.502
- 0.480
- **0.498**
- 0.300

Approach: I am training using Stochastic Gradient Descent. My current weight = 0.5, gradient = 0.2, and learning rate = 0.01. So, my new weight = $\text{weight} - (\text{gradient} \times \text{learning rate})$. That is, new weight = $0.5 - (0.2 \times 0.01) = 0.498$.

Question 8

Problem: You want to apply a convolution with `kernel_size=5` and `stride=1` such that the output size remains the same as the input. What padding value must you set?

Options:

- 1
- 2
- 3
- 5

Approach: I am applying `kernel_size=5` and `stride=1`. Meaning my output dimension will decrease by $(\text{kernel_size} - 1)$ compared to the input dimension, as the stride is 1. To ensure that the output dimension stays the same as the input, I will increase padding by $(\text{kernel_size} - 1)/2$, which gives `padding=2`.

Question 9

Problem: In a PyTorch training loop using wandb, which command is used to send the current loss and accuracy to the dashboard?

Options:

- `wandb.save({"loss": loss})`
- `wandb.log({"loss": loss, "acc": accuracy})`
- `wandb.track(loss)`
- `wandb.print(loss)`

Approach: Usually, most developers use a logging command (like `wandb.log()`) to send any parameters they want to the dashboard for visualizing the training progress.

Question 10

Problem: After training, you evaluate your model on a test set of 200 spectrograms and it correctly predicts 170 of them. What is the accuracy?

Options:

- 0.75
- 0.80
- 0.85
- 0.90

Approach: Accuracy is the ratio of correct predictions to total predictions. That is $170/200 = 0.85$. Hence, 0.85 is the accuracy.